

Versi 2.0

Januari 2026



PRAKTIKUM SISTEM OPERASI

MODUL 4 - PENDAHULUAN TENTANG PROSES MANAJEMEN

Pengarang:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

LABORATORIUM INFORMATIKA

Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KATA KUNCI.....	3
LATAR BELAKANG TEORI.....	4
1. DASAR-DASAR PROSES.....	4
1.1. Definisi dan siklus hidup suatu proses.....	4
LATIHAN	
1.....	5
1.2. Status proses dan Blok Kontrol Proses (PCB).....	5
1.3. Hubungan induk-anak, Proses yatim piatu, dan Proses zombie.....	5
LATIHAN	
2.....	8
2. MANAJEMEN PROSES DI LINUX.....	8
2.1. Mengidentifikasi proses: PID, 'ps', 'top', DAN 'pstree'.....	8
LATIHAN	
3.....	10
2.2. Kontrol Pekerjaan: Latar Depan, Latar Belakang, dan Penangguhan.....	10
LATIHAN	
4.....	11
2.3. Debugging Praktis: menggunakan 'systemctl status' dan 'journalctl' untuk kegagalan layanan.....	11
KONTROL DAN PENGHENTIAN PROSES.....	13
3.1. Sinyal dan Penghentian Proses ('kill', 'pkill', 'SIGTERM', 'SIGKILL').....	13
LATIHAN	
5.....	15
3.2. Penanganan Proses yang Aman: menghindari penghentian paksa bila memungkinkan.....	15
4. PENJADWALAN CPU DAN ANALISIS KINERJA.....	17
4.1. Multiprogramming, Multitasking, dan kebutuhan akan Penjadwalan.....	17
4.2. Algoritma inti: FCFS, SJF, Round Robin, Penjadwalan Prioritas.....	18
4.3. Metrik kinerja: bagan Gantt, waktu tunggu, waktu penyelesaian.....	19
APLIKASI	
PRAKTIS.....	21
LATIHAN.....	21
DEMO - STUDI KASUS.....	21
RUBRIK PENILAIAN.....	23
SKALA PENILAIAN.....	23



LATAR BELAKANG TEORI

1. DASAR-DASAR PROSES Dalam sistem

operasi modern seperti **Rocky Linux**, kemampuan untuk menjalankan banyak program secara bersamaan bukanlah sihir; itu adalah hasil dari manajemen proses yang cermat oleh kernel. **Proses** adalah unit kerja fundamental dalam sistem operasi apa pun. Memahami apa itu proses, bagaimana perilakunya, dan bagaimana interaksinya dengan proses lain sangat penting untuk **administrasi sistem** dan **pengembangan perangkat lunak**.

Bagian ini memperkenalkan konsep-konsep teoretis inti yang mendasari **perilaku proses dalam sistem berbasis Linux**, meletakkan dasar untuk pemantauan, pengendalian, dan analisis kinerja praktis di bagian selanjutnya.

1.1. Definisi dan siklus hidup suatu proses A. Apa Itu Proses?

Suatu **proses** adalah contoh dari sebuah program yang sedang dieksekusi. Sementara program adalah kumpulan instruksi pasif yang tersimpan di disk (misalnya, `/bin/ls`), proses adalah entitas aktif yang dibuat ketika program tersebut dimuat ke dalam memori dan dieksekusi oleh CPU.

Sebuah program tunggal (seperti `'chrome'`) dapat menjalankan beberapa proses (misalnya, satu untuk jendela utama, yang lain untuk pembaruan latar belakang). Setiap proses memiliki ruang memori, register, dan status eksekusinya sendiri.

Di Linux, setiap proses diberi ID Proses (PID) unik oleh kernel. PID ini digunakan oleh sistem dan administrator untuk mengidentifikasi, memantau, dan mengontrol proses sepanjang masa hidupnya.

B. Siklus Hidup Proses

Suatu proses melewati serangkaian tahapan yang terdefinisi dengan baik dari pembuatan hingga penghentian, yang dikenal

- sebagai **siklus hidupnya**: 1. **Baru**: Proses sedang dibuat (misalnya, melalui `'fork()'` atau eksekusi shell).
2. **Siap**: Proses dimuat ke dalam memori dan menunggu CPU untuk mengeksekusinya.
3. **Sedang Berjalan**: Proses saat ini sedang dieksekusi oleh CPU.
4. **Menunggu (atau Terblokir)**: Proses dihentikan sementara, menunggu suatu kejadian (misalnya, masukan pengguna).
5. **Dihentikan**: Proses telah selesai dieksekusi atau dihentikan secara paksa.

Siklus hidup ini dikelola sepenuhnya oleh kernel sistem operasi, yang memastikan penggunaan waktu CPU yang adil dan efisien di antara semua proses aktif.



Contoh:

Saat Anda mengetik 'ping google.com' di terminal: • Sebuah proses baru dibuat ('Baru') • Proses tersebut masuk ke antrian siap ('Siap') • CPU mengeksekusinya ('Berjalan') • Proses tersebut menunggu di antara ping ('Menunggu') • Anda menekan 'Ctrl+C', dan proses tersebut dihentikan ('Dihentikan')

LATIHAN 1

Jalankan perintah `ping -c 5 google.com`, kemudian amati tahapan siklus hidup prosesnya (Baru → Berjalan → Menunggu → Dihentikan) menggunakan `ps aux | grep ping` pada saat yang sama ketika ping masih berjalan. (Tips: gunakan 2 terminal secara bersamaan)

1.2. Status proses dan Blok Kontrol Proses (PCB)**A. Status Proses** Pada

setiap saat tertentu, suatu proses berada dalam salah satu dari lima status yang dijelaskan di atas. Sistem operasi memelihara informasi status ini untuk mengkoordinasikan alokasi sumber daya dan penjadwalan CPU.

Transisi antar status dipicu oleh peristiwa seperti: • **Panggilan sistem**

(misalnya, 'read()', 'write()') • **Interupsi** (misalnya, interupsi timer, input keyboard) • **Sinyal** (misalnya, 'SIGTERM', 'SIGKILL')

Transisi ini memastikan bahwa tidak ada satu proses pun yang memonopoli sumber daya sistem—prinsip utama dari sistem operasi multitasking.

B. Blok Kontrol Proses (PCB)

Process **Control Block (PCB)** adalah struktur data penting yang dikelola oleh kernel untuk setiap proses. Ia bertindak sebagai "kartu identitas" suatu proses, menyimpan semua informasi yang diperlukan untuk mengelola dan melanjutkan eksekusinya.

Komponen utama PCB meliputi:

Bidang	Keterangan
Status Proses	Status saat ini (misalnya, Berjalan, Menunggu)
Penghitung Program (PC)	Alamat instruksi berikutnya yang akan dieksekusi
Register CPU	Cuplikan nilai register (untuk peralihan konteks)
Informasi Penjadwalan CPU	Prioritas, penunjuk antrian penjadwalan
Informasi Manajemen Memori	Register dasar/batas, tabel halaman
Informasi Akuntansi	Waktu CPU yang digunakan, batas waktu, nomor proses



Informasi Status Input/Output	Daftar file yang terbuka, perangkat yang dialokasikan
ID Proses (PID)	Pengidentifikasi numerik unik
PID Induk (PPID)	PID dari proses yang membuatnya

Ketika sistem operasi melakukan **peralihan konteks** (beralih dari satu proses ke proses lain), ia menyimpan status proses saat ini ke dalam PCB-nya dan memuat status proses berikutnya dari PCB-nya ke dalam register CPU.

Mengapa Ini Penting:

Tanpa PCB, sistem operasi tidak dapat menangguhkan dan kemudian melanjutkan proses dengan akurat, sehingga multitasking menjadi tidak mungkin.

1.3. Hubungan Induk-Anak, Proses Yatim, dan Proses Zombie A. Proses Induk dan Anak Dalam

sistem mirip Unix seperti Rocky Linux, proses

diatur dalam **struktur pohon hierarkis**. Ketika suatu proses membuat proses baru (biasanya menggunakan panggilan sistem

'fork()'), proses tersebut menjadi **induk**, dan proses baru tersebut menjadi **anak**.

- Setiap proses (kecuali yang pertama, 'init' atau 'systemd') memiliki tepat satu induk.
- Sebuah induk dapat memiliki banyak anak.
- Hubungan tersebut dilacak melalui **ID Proses Induk**

(**PPID**) yang tersimpan di PCB setiap anak.

Anda dapat melihat hierarki ini menggunakan:

```
praktikumso@localhost:~$ pstree
systemd--ModemManager--3*[{ModemManager}]
      |--NetworkManager--3*[{NetworkManager}]
      |--SetroubleshootP--3*[{SetroubleshootP}]
      |--accounts-daemon--3*[{accounts-daemon}]
```

atau

```
praktikumso@localhost:~$ ps -ef
UID      PID     PPID  C  STIME TTY
root         1         0  0  17:58 ?
root         2         0  0  17:58 ?
root         3         2  0  17:58 ?
```



B. Proses Anak Yatim Proses

anak **yatim** adalah proses anak yang orang tuanya telah mengakhiri hak warisnya sebelum anak tersebut.

- Dalam kebanyakan kasus, proses yatim piatu secara otomatis **diadopsi oleh proses 'init'** (PID 1, atau 'systemd' di Linux modern).
- Hal ini memastikan bahwa setiap proses selalu memiliki induk, sehingga menjaga integritas sistem.

Catatan: **Terisolasi** y

Masalah: Terisolasi adalah hal normal dan aman. Misalnya, daemon latar belakang sering kali sengaja terisolasi agar dapat berjalan secara independen.

C. Proses Zombie

Proses **zombie** (atau proses yang sudah tidak berfungsi) adalah proses yang telah **menyelesaikan eksekusi** tetapi masih memiliki entri dalam tabel proses karena induknya belum membaca status keluarnya.

- Proses zombie tidak mengonsumsi CPU atau memori, tetapi mereka **menempati PID**.
- Jika terlalu banyak yang terakumulasi, sistem mungkin kehabisan PID (meskipun ini jarang terjadi).

Bagaimana Zombie Terbentuk:

1. Proses anak selesai dan memanggil **'exit()'**.
2. Kernel menyimpan informasi minimal (status keluar) di PCB.
3. Induk harus memanggil **'wait()'** atau **'waitpid()'** untuk mengumpulkan status ini.
4. Sampai saat itu, anak tersebut tetap menjadi zombie.

Mengidentifikasi Zombie:

Pada output 'ps', zombie akan muncul dengan huruf **'Z'** di kolom STAT:

```
praktikumso@localhost:~$ ps aux | grep 'Z'
```

Catatan:

Zombie y Tidak Berbahaya Selamanya:

Meskipun beberapa zombie adalah hal normal, zombie yang terus-menerus muncul menunjukkan adanya bug pada proses induk (misalnya, lupa memanggil **'wait()'**).

Ringkasan:

Proses adalah jantung dari setiap sistem operasi. Dari saat perintah diketik di terminal hingga saat layanan berjalan di latar belakang, kernel mengelola masing-masing sebagai proses yang berbeda dengan siklus hidup, status, dan identitas yang ditentukan. Memahami PCB (Process Control Block), hubungan induk-anak, dan keadaan abnormal seperti zombie mempersiapkan Anda untuk men-debug masalah sistem dunia nyata seperti layanan yang tidak responsif atau kebocoran sumber daya menggunakan alat seperti **'ps'**, **'top'**, dan **'pstree'**.



LATIHAN 2

Gunakan perintah `ps -ef` untuk menemukan PID dan PPID dari proses bash Anda, lalu bandingkan dengan output dari `pstree -p` untuk memvisualisasikan hubungan induk-anak.

2. MANAJEMEN PROSES DI LINUX

Pada sistem berbasis Linux seperti Rocky Linux, setiap program yang berjalan, mulai dari perintah `ls` sederhana hingga server web yang kompleks, direpresentasikan sebagai sebuah **proses**. Kernel sistem operasi menetapkan pengidentifikasi unik untuk setiap proses dan mengelola eksekusi, memori, dan interaksinya dengan proses lain. Memahami cara **mengidentifikasi**, **memantau**, dan **mengontrol** proses-proses ini sangat penting baik untuk administrasi sistem maupun pemecahan masalah.

Bagian ini memperkenalkan alat dan teknik praktis yang digunakan sehari-hari oleh administrator Linux untuk mengamati dan mengelola proses aktif langsung dari terminal tanpa bergantung pada antarmuka grafis.

2.1. Mengidentifikasi proses: PID, 'ps', 'top', DAN 'pstree'

A. Apa itu PID?

Setiap proses di Linux diberi **ID Proses (PID)**, yaitu bilangan bulat positif unik yang membedakannya dari semua proses lain. Proses pertama yang dijalankan oleh kernel adalah 'systemd' (atau 'init' pada sistem yang lebih lama), yang selalu memiliki PID 1. Semua proses lain merupakan turunan dari proses akar ini.

Anda dapat melihat PID shell Anda saat ini menggunakan:

```
praktikumso@localhost:~$ echo $$  
3234
```

B. Perintah 'ps' : Cuplikan Proses yang Aktif

Perintah 'ps' (status proses) memberikan **gambaran statis** tentang proses yang sedang berjalan. Ini adalah alat paling mendasar untuk identifikasi proses.

Penggunaan umum:

```
praktikumso@localhost:~$ ps aux
```

- `'a'` ÿ menampilkan proses untuk semua pengguna
- `'u'` ÿ menampilkan pemilik proses dan penggunaan sumber daya
- `'x'` ÿ menyertakan proses yang tidak terhubung ke terminal



Contoh Interpretasi Keluaran:

USER	PID	%CPU	%MEM	VSZ	RSS	TTY
root	1	0.0	1.1	50180	41872	?
root	2	0.0	0.0	0	0	?
root	3	0.0	0.0	0	0	?

• **PID 1234:** Proses 'vim' yang dimiliki oleh 'praktikumso' • **STAT "S+":**

Tidur ('S') dan di latar depan ('+') • **%CPU / %MEM:** Penggunaan CPU dan memori

Tip: Gunakan '**ps -ef**' untuk format yang sesuai dengan POSIX yang menyertakan PPID (Parent PID).

C. Perintah '**top**' dan '**htop**': Pemantauan Waktu Nyata Sementara 'ps'

memberikan tampilan sekali saja, 'top' menyediakan **tampilan dinamis dan waktu nyata** dari proses sistem, yang diurutkan berdasarkan penggunaan CPU secara default.

Untuk memulai:

```
praktikumso@localhost:~$ top
```

Fitur utama: •

Terus diperbarui setiap beberapa detik • Menampilkan total CPU, memori, dan rata-rata beban • Memungkinkan perintah interaktif (misalnya, 'k' untuk menghentikan proses)

Untuk antarmuka yang lebih ramah pengguna, instal 'htop':

```
praktikumso@localhost:~$ sudo dnf install htop -y
```

```
praktikumso@localhost:~$ htop
```

'htop'

menambahkan: • Output berkode warna • Pengguliran vertikal dan horizontal • Dukungan mouse (jika diaktifkan) • Tampilan pohon hubungan induk-anak

Praktik Terbaik: Gunakan '**top**' untuk pengecekan cepat; gunakan '**htop**' untuk analisis lebih mendalam selama praktikum.

D. Perintah '**ps**tree': Memvisualisasikan Hierarki Proses

Proses di Linux membentuk **struktur pohon**, di mana setiap proses anak mewarisi dari proses induknya. Perintah '**ps**tree' menampilkan hierarki ini secara visual.



Contoh:

```
praktikumso@localhost:~$ pstree -p
```

Cuplikan output:

```
systemd(1)─sshd(890)─sshd(1200)─bash(1201)─vim(1234)─httpd(900)─httpd(901)
```

httpd(902) • PID ditampilkan dalam

tanda kurung •

Jelas menunjukkan bahwa 'vim(1234)'

adalah anak dari 'bash(1201)', yang berada di bawah 'sshd'

Catatan: Ini sangat berharga untuk mengidentifikasi rantai proses yang terisolasi atau berperilaku tidak wajar.

LATIHAN 3

Jalankan perintah `top`, tekan `1` untuk melihat penggunaan CPU per inti, kemudian identifikasi proses dengan beban tertinggi dan catat %CPU, PID, dan statusnya (S, R, dll.).

2.2. Kontrol Pekerjaan: Foreground, Background, dan Suspension Shell Linux

(seperti Bash) mendukung **kontrol pekerjaan**, memungkinkan pengguna untuk mengelola beberapa proses dalam satu sesi terminal.

A. Menjalankan Tugas di Latar Belakang

Tambahkan '&' ke perintah untuk menjalankannya di latar belakang:

```
praktikumso@localhost:~$ sleep 60 &
```

Keluaran:

```
[2] 8217
```

• '[2]' = nomor pekerjaan •

'8217' = PID dari proses latar belakang

Anda dapat terus menggunakan terminal untuk menjalankan perintah lain saat proses sedang berjalan.

B. Menampilkan dan Mengelola Tugas

Gunakan perintah 'jobs' untuk menampilkan semua tugas di shell saat ini:

```
praktikumso@localhost:~$ jobs
[1]+  Stopped                  vim test.sh
[2]-  Done                     sleep 60
```

Memindahkan Pekerjaan Antara Latar Depan dan Latar Belakang

• Prioritaskan pekerjaan:

```
praktikumso@localhost:~$ fg %1
vim test.sh
```



- Menghentikan sementara proses latar depan yang sedang berjalan: Tekan 'Ctrl + Z'. Ini akan mengirimkan sinyal 'SIGTSTP' dan menghentikan sementara proses tersebut.
- Melanjutkan pekerjaan yang ditangguhkan di latar belakang:

```
praktikumso@localhost:~$ bg %1
```

Catatan: Kontrol pekerjaan hanya berfungsi untuk proses yang dimulai dari **sesi shell saat ini**. Ini tidak memengaruhi layanan sistem seperti 'sshd' atau 'httpd'.

C. Contoh Praktis: Mengelola Naskah yang Berjalan Lama

1. Buat skrip:

```
praktikumso@localhost:~$ echo 'for i in {1..10}; do echo "step $i"; sleep 2; done' > longtask.sh
praktikumso@localhost:~$ chmod +x longtask.sh
```

2. Jalankan di latar belakang:

```
praktikumso@localhost:~$ ./longtask.sh &
[1] 8470
```

3. Periksa lowongan pekerjaan:

```
praktikumso@localhost:~$ jobs
```

4. Hentikan dan lanjutkan:

```
praktikumso@10:~$ fg %1
```

Tekan Ctrl+Z

```
praktikumso@10:~$ bg %1
```

5. Hentikan jika diperlukan:

```
praktikumso@10:~$ kill %1
```

LATIHAN 4

Jalankan perintah `sleep 30 &`, lalu gunakan `jobs`, `fg`, dan `Ctrl+Z` untuk memindahkan proses antara status foreground, background, dan suspended.

2.3. Debugging Praktis: menggunakan 'systemctl status' dan 'journalctl' untuk kegagalan layanan

Pada distribusi Linux modern seperti Rocky Linux, sebagian besar layanan sistem (misalnya, SSH, Apache, MySQL) dikelola oleh **systemd**. Ketika suatu layanan gagal dimulai, dua perintah kunci dapat membantu mendiagnosis masalah tersebut.



A. Langkah 1: Periksa Status Layanan dengan 'systemctl status'

Contoh: Melakukan debugging pada layanan SSH yang gagal setelah mengubah port-nya.

```
praktikumso@10:~$ sudo systemctl status sshd
```

Output tipikal meliputi:

- Apakah layanan **aktif (berjalan)** atau **tidak aktif (mati)**
- **PID utama** (jika berjalan)
- **Baris log terbaru** yang menunjukkan beberapa kejadian terakhir
- Kode keluar dan pesan kesalahan (misalnya, "Alamat sudah digunakan")

Catatan: Jika layanan gagal, cari baris atau pesan berwarna merah seperti: Gagal memulai daemon server **OpenSSH**.

B. Langkah 2: Periksa Log Terperinci dengan 'journalctl'

Perintah '**journalctl**' mengakses **jurnal systemd**, yang menyimpan log terstruktur untuk semua layanan.

- Untuk melihat log 'sshd':

```
praktikumso@10:~$ sudo journalctl -u sshd
```

- Untuk pemantauan waktu nyata:

```
praktikumso@10:~$ sudo journalctl -u sshd -f
```

- Untuk melihat hanya 20 baris terakhir:

```
praktikumso@10:~$ sudo journalctl -u sshd --since "5 minutes ago"
```

Pola Kesalahan Umum:

Pesan Kesalahan	Kemungkinan Penyebab
Alamat sudah digunakan	Konflik port (misalnya, aplikasi lain menggunakan port 2222)
Izin ditolak	SELinux memblokir port
Kesalahan file konfigurasi	Kesalahan sintaksis di /etc/ssh/sshd_config

C. Langkah 3: Perbaiki dan

Mulai Ulang Setelah

- mengidentifikasi masalah: 1. Edit file konfigurasi (misalnya, '**/etc/ssh/sshd_config**')
2. Uji sintaks:

```
praktikumso@10:~$ sudo ssh -t
```

3. Jika valid, mulai ulang:

```
praktikumso@10:~$ sudo systemctl restart sshd
```



Selalu verifikasi:

```
praktikumso@10:~$ sudo systemctl restart sshd
```

D. Mengapa Hal Ini Penting bagi Siswa

- Server di dunia nyata jarang memiliki GUI—debugging dilakukan di terminal.
- Memahami 'systemctl' dan 'journalctl' sangat penting untuk DevOps, peran cloud, dan Administrasi sistem.
- Alat-alat ini merupakan standar di RHEL, CentOS, Fedora, dan Rocky Linux.

Tip: Gabungkan dengan 'grep' untuk penyaringan

yang lebih cepat: `journalctl -u sshd | grep "Failed"`

Ringkasan:

Linux menyediakan alat berbasis CLI yang ampuh untuk **mengamati**, **mengontrol**, dan **men-debug** proses. Dari perintah sederhana seperti 'ps' hingga inspeksi log tingkat lanjut dengan 'journalctl', menguasai utilitas ini mempersiapkan Anda untuk mengelola sistem nyata dengan percaya diri persis seperti yang diharapkan di lingkungan profesional.

3. KONTROL DAN PENGHENTIAN PROSES

Di Linux, proses tidak berjalan tanpa batas; proses tersebut pada akhirnya harus berakhir, baik dengan menyelesaikan tugasnya atau dihentikan oleh sistem atau pengguna. Sistem operasi menyediakan mekanisme yang tepat untuk mengontrol, menginterupsi, dan mengakhiri proses dengan aman. Mekanisme ini didasarkan pada sinyal, suatu bentuk komunikasi antar proses yang memungkinkan kernel atau proses lain untuk memberi tahu suatu proses tentang suatu peristiwa.

Memahami cara mengirim sinyal dan kapan harus menggunakan penghentian yang lembut versus penghentian paksa sangat penting untuk administrasi sistem dan debugging yang bertanggung jawab.

3.1. Sinyal dan Penghentian Proses ('kill', 'pkill', 'SIGTERM', 'SIGKILL')

A. Apa Itu Sinyal?

Sinyal adalah interupsi perangkat lunak yang dikirimkan ke suatu proses untuk memberitahunya tentang suatu peristiwa penting. Sinyal

- dapat berasal dari:
- Kernel (misalnya, kesalahan segmentasi),
 - Proses lain (misalnya, proses induk menghentikan proses anak),
 - Pengguna (misalnya, menekan 'Ctrl+C').



Setiap sinyal memiliki **nama** dan **nomor**. Sinyal umum meliputi:

Sinyal	Nomor	Tindakan Default	Keterangan
SIGTERM	15	Akhiri dengan elegan.	Permintaan untuk mematikan sistem dengan bersih.
SIGKILL	9	Bunuh paksa (tidak bisa) tertangkap atau diabaikan)	Pemutusan hubungan kerja segera
SINYAL	2	Mengakhiri	Dikirim dengan 'Ctrl+C'
SIGSTOP	19	Berhenti sebentar	Menangguhkan proses (tidak bisa) (diabaikan)
SIGCONT	18	Melanjutkan	Lanjutkan proses yang dihentikan sementara

Pemahaman inti:

Hanya 'SIGKILL' dan 'SIGSTOP' yang tidak dapat ditangani, diblokir, atau diabaikan oleh suatu proses. Semua yang lain dapat dideteksi dan ditanggapi sehingga memungkinkan program untuk melakukan pembersihan sebelum dihentikan.

B. Perintah 'bunuh'

Terlepas dari namanya, 'kill' tidak selalu menghentikan suatu proses, melainkan mengirimkan sinyal.

Sintaks dasar:

```
praktikumso@10:~$ kill [signal] PID
```

Contoh:

Kirim SIGTERM (default)

```
praktikumso@10:~$ kill 1234
```

Kirim SIGKILL (bunuh paksa)

```
praktikumso@10:~$ kill -9 1234
```

Kirim SIGTERM secara eksplisit

```
praktikumso@10:~$ kill -15 1234
```

Praktik Terbaik: Selalu coba 'SIGTERM' terlebih dahulu. Gunakan 'SIGKILL' hanya jika proses tidak merespons.

Perintah 'pkill' dan 'killall'

Jika Anda tidak mengetahui PID, Anda dapat menargetkan proses berdasarkan nama:

- 'pkill': Mengirimkan sinyal ke proses yang cocok dengan suatu pola.

```
praktikumso@10:~$ pkill firefox
```

```
praktikumso@10:~$ pkill -f "python script.py"
```

- 'killall': Menghentikan semua proses dengan nama tertentu.



```
praktikumso@10:~$ killall httpd
```

Perhatian: Perintah 'killall' pada beberapa sistem (seperti Solaris) menghentikan **semua proses**—tetapi di Linux, perintah ini hanya memengaruhi proses berdasarkan namanya. Meskipun demikian, gunakan dengan hati-hati.

C. Contoh Praktis: Pemutusan Hubungan Kerja Secara Baik vs. Secara Paksa

1. Memulai proses yang berlangsung lama:

```
praktikumso@10:~$ sleep 300 &
[1] 10293
```

Catat PID-nya (misalnya, '10293').

2. Coba lakukan penghentian yang elegan:

```
praktikumso@10:~$ kill 10293
[1]+  Terminated                  sleep 300
```

↳ Proses 'sleep' langsung keluar (proses ini menangani 'SIGTERM').

3. Sekarang mulailah yang lain:

```
praktikumso@10:~$ sleep 300 &
[1] 10334
```

4. Jika tidak responsif, bunuh secara paksa:

```
praktikumso@10:~$ kill -9 10334
[1]+  Killed                        sleep 300
```

Gunakan 'ps' setelahnya untuk memastikan proses telah selesai.

LATIHAN 5

Jalankan perintah `sleep 60 &`, catat PID-nya, kirim sinyal `SIGTERM` menggunakan `kill`, dan verifikasi apakah proses tersebut berhenti. Jika proses tersebut tetap berjalan, hentikan secara paksa menggunakan `SIGKILL`.

3.2. Penanganan Proses yang Aman: menghindari pembunuhan paksa jika memungkinkan

A. Mengapa Harus Menghindari 'SIGKILL'?

Menggunakan 'kill -9' mungkin tampak efektif, tetapi ada risikonya: • **Tidak ada pembersihan:**

File yang terbuka, koneksi jaringan, atau data sementara mungkin tidak ditutup.
dengan baik.

• **Kerusakan data:** Basis data atau file log mungkin berada dalam keadaan tidak konsisten. • **Pengalaman pengguna yang buruk:** Aplikasi tidak dapat menyimpan status sesi atau memberi tahu pengguna.

Aturan praktis:

Jangan pernah memulai dengan 'SIGKILL'. Selalu coba matikan sistem secara bertahap terlebih dahulu.



B. Alur Kerja Penghentian yang Direkomendasikan

1. Identifikasi prosesnya: `ps`

`aux | grep nginx` ATAU

`pgrep nginx` 1. Kirim

'SIGTERM' (permintaan yang baik): `kill $(pgrep nginx)`

2. Tunggu 5–10 detik. Periksa apakah sudah hilang:

`pgrep nginx` 3.

Hanya jika masih berjalan, kirim 'SIGKILL': `pkill -9`

`nginx`

Pendekatan ini digunakan oleh layanan sistem seperti '`systemctl stop nginx`'.

C. Studi Kasus Dunia Nyata: Menghentikan Server Web

Misalkan Apache ('`httpd`') mengalami masalah.

Cara yang benar:

```
praktikumso@10:~$ sudo systemctl stop httpd
```

Perintah ini mengirimkan 'SIGTERM', menunggu, lalu 'SIGKILL' jika diperlukan.

Cara yang salah:

```
praktikumso@10:~$ sudo kill -9 $(pgrep httpd)
```

Dapat menyebabkan soket tetap terbuka atau log tidak lengkap.

D. Penanganan Sinyal dalam Skrip

Anda juga dapat menangkap sinyal dalam skrip Bash untuk melakukan pembersihan:

```
1 #!/bin/bash
2
3 cleanup(){
4     echo "Shutting down gracefully..."
5     rm -f /tmp/myapp.lock
6     exit 0
7 }
8
9 #Trap SIGTERM and SIGINT
10 trap cleanup TERM INT
11
12 # Simulate work
13 while true: do
14     sleep 1
15 done
```

Sekarang, 'kill [PID]' atau 'Ctrl+C' akan memicu fungsi 'pembersihan'.

Ini adalah praktik profesional yang digunakan dalam daemon produksi.



Ringkasan:

Penghentian proses bukan hanya tentang menghentikan program, tetapi juga tentang **melakukannya secara bertanggung jawab**. Dengan memahami sinyal dan menggunakan **'SIGTERM'** sebelum **'SIGKILL'**, Anda memastikan stabilitas sistem, integritas data, dan administrasi tingkat profesional. Keterampilan ini dapat diterapkan langsung saat mengelola layanan seperti **'sshd'**, **'httpd'**, atau skrip kustom di Rocky Linux.

4. PENJADWALAN CPU DAN ANALISIS KINERJA

Dalam sistem operasi modern seperti Rocky Linux, banyak proses sering kali bersaing untuk mendapatkan sumber daya CPU yang terbatas. Tanpa cara sistematis untuk memutuskan proses mana yang akan berjalan selanjutnya, sistem akan menjadi tidak efisien, tidak responsif, atau bahkan tidak adil. Di sinilah penjadwalan CPU berperan.

Penjadwalan CPU adalah mekanisme yang digunakan kernel sistem operasi untuk memilih proses selanjutnya yang akan dieksekusi pada CPU. Hal ini memastikan bahwa sumber daya sistem dibagi secara adil, waktu respons diminimalkan, dan throughput dimaksimalkan—terutama dalam lingkungan multi-pengguna atau server.

Bagian ini memperkenalkan konsep-konsep dasar penjadwalan, mengeksplorasi algoritma inti yang digunakan dalam praktik dan pendidikan, serta menyediakan alat untuk mengevaluasi kinerja algoritma tersebut secara kuantitatif menggunakan metrik standar.

4.1. Multiprogramming, Multitasking, dan kebutuhan akan Penjadwalan

A. Apa Itu Multiprogramming?

Multiprogramming adalah teknik menyimpan beberapa proses dalam memori secara bersamaan.

Ketika suatu proses sedang menunggu (misalnya, untuk I/O), CPU dapat beralih ke proses lain yang siap—sehingga memaksimalkan pemanfaatan CPU.

Contoh: Saat skrip pencadangan menunggu penulisan ke disk, sistem masih dapat merespons permintaan login SSH.

B. Apa Itu Multitasking?

Multitasking memperluas kemampuan multiprogramming dengan beralih antar proses secara cepat, memberikan ilusi kepada pengguna bahwa banyak program berjalan **secara bersamaan**, bahkan pada CPU inti tunggal.

- Di Linux, hal ini dicapai melalui **pembagian waktu**: setiap proses mendapatkan sebagian kecil dari CPU. waktu.
- Kernel sistem operasi menggunakan penjadwal untuk mengelola peralihan ini.

C. Mengapa Penjadwalan Diperlukan?

Tanpa penjadwalan: •

- Satu proses yang membebani CPU dapat memonopoli CPU tanpa batas waktu.
- Aplikasi interaktif (seperti terminal atau server web) akan menjadi tidak responsif.



- Keadilan dan efisiensi sistem akan runtuh.

Oleh karena itu, peran penjadwal sangat penting: ia menyeimbangkan **throughput**, **latensi**, **keadilan**, dan **pemanfaatan sumber daya**.

4.2. Algoritma inti: FCFS, SJF, Round Robin, Penjadwalan Prioritas

Terdapat beberapa algoritma penjadwalan, masing-masing dengan kelebihan dan kekurangannya. Di bawah ini adalah empat algoritma fundamental yang dibahas dalam RPS dan umum digunakan dalam konteks akademis dan dunia nyata.

A. Siapa Cepat Dia Dapat (FCFS)

- **Prinsip:** Proses dieksekusi sesuai urutan kedatangannya.
- **Tipe:** Non-preemptif

Kelebihan: Mudah diimplementasikan.

- **Kekurangan:** Dapat menyebabkan **efek**

konvoi, proses pendek menunggu di belakang proses yang panjang.

Contoh: •

Proses tiba: P1 (10 ms), P2 (2 ms), P3 (5 ms) • Urutan eksekusi:

P1 → P2 → P3 • P2 dan P3 menunggu tanpa

perlu.

B. Pekerjaan Terpendek Pertama (SJF)

- **Prinsip:** Proses dengan **waktu eksekusi** terpendek (waktu CPU yang dibutuhkan) dijalankan terlebih dahulu.
- **Tipe:**

Dapat berupa **non-preemptif** (SJF) atau **preemptif** (Shortest Remaining Time First,

SRTF).

- **Kelebihan:** Meminimalkan waktu tunggu rata-rata (optimal jika waktu eksekusi diketahui).
- **Kekurangan:**

Membutuhkan pengetahuan sebelumnya tentang waktu eksekusi; dapat menyebabkan **kelaparan (starvation)** pada proses yang panjang.

Untuk analogi di dunia nyata: Seorang

kasir melayani pelanggan dengan barang yang lebih sedikit terlebih dahulu.

C. Sistem Round Robin (RR)

- **Prinsip:** Setiap proses mendapatkan potongan waktu tetap yang disebut **kuantum waktu** (misalnya, 2 ms atau 5 ms)

ms). •

Tipe: Preemptif • **Kelebihan:**

Adil; tidak terjadi kelaparan; baik untuk sistem interaktif. • **Kekurangan:** Overhead

peralihan konteks yang tinggi jika kuantum terlalu kecil; pemanfaatan CPU yang buruk jika

terlalu besar.

Di Linux:

Penjadwal default (CFS – Completely Fair Scheduler) menggunakan varian RR dinamis dan berbagi adil.



D. Penjadwalan Prioritas

1. **Prinsip:** Setiap proses diberi prioritas; proses dengan prioritas tertinggi dijalankan terlebih dahulu.
2. **Tipe:** Bisa preemptif atau non-preemptif.
3. **Kelebihan:** Berguna untuk tugas-tugas real-time atau kritis.
4. **Kelemahan:** Proses dengan prioritas rendah mungkin akan **mengalami kelaparan** tanpa batas waktu.

Solusi: **Penuaan secara bertahap** meningkatkan prioritas proses menunggu untuk mencegah kelaparan.

Nilai prioritas: •

Angka yang lebih rendah = prioritas yang lebih tinggi (misalnya, 0 = tertinggi, 10 = terendah) • Atau sebaliknya, tergantung pada desain sistem.

4.3. Metrik kinerja: bagan Gantt, waktu tunggu, waktu penyelesaian

Untuk membandingkan algoritma penjadwalan secara objektif, kami menggunakan metrik kuantitatif.

A. Bagan Gantt

Bagan **Gantt** secara visual merepresentasikan garis waktu pelaksanaan proses dari waktu ke waktu.

Contoh (FCFS): |----

P1----|---**P2**--|-----**P3**-----| 0 17
10 12

Bagan Gantt membantu memvisualisasikan: •

Waktu idle CPU • Periode tunggu proses • Titik preemption (dalam RR atau SRTF)

B. Waktu Tunggu (WT)

Waktu Tunggu = Total waktu yang dihabiskan suatu proses dalam **antrian siap** sebelum mendapatkan CPU.

Rumus:

$$WT = \text{Waktu Penyelesaian} - \text{Waktu Eksekusi}$$

Tujuan: Meminimalkan waktu tunggu rata-rata di semua proses.

C. Waktu Penyelesaian (TAT)

Waktu Penyelesaian = Total waktu dari **saat proses dimulai** hingga **selesai**.

Rumus:

$$TAT = \text{Waktu Penyelesaian} - \text{Waktu Kedatangan}$$

Tujuan: Meminimalkan waktu penyelesaian rata-rata untuk meningkatkan responsivitas sistem.



D. Menghitung Metrik Rata-Rata

Diberikan sekumpulan proses:

Proses	Waktu Ledakan	Waktu Kedatangan
P1	10	0
Halaman 2	2	3
P3	5	4

Untuk FCFS (First Come First Serve):

- P1: $TAT = 10 - 0 = 10$, $WT = 10 - 10 = 0$
- P2: $TAT = 12 - 3 = 9$, $WT = 9 - 2 = 7$
- P3 : $TAT = 17 - 4 = 13$, $WT = 13 - 5 = 8$

• Rata-rata $WT = (0 + 7 + 8) / 3 = 5,0$

• Rata-rata $TAT = (10 + 9 + 13) / 3 = 10,67$

E. Koneksi Praktis ke Rocky Linux

Meskipun Anda tidak akan menerapkan FCFS atau SJF secara manual di Rocky Linux (kernel menggunakan CFS), Memahami algoritma ini akan membantu Anda:

- Menginterpretasikan output 'top'/'htop' ,
- Sesuaikan prioritas proses dengan 'nice' dan 'renice',
- Mendiagnosis hambatan kinerja dalam skrip atau layanan.

Perintah untuk melihat prioritas proses:

```
praktikumso@10:~$ ps -eo pid,comm,pri,ni
```

- 'ni' = nilai bagus (nilai lebih rendah = prioritas lebih tinggi)

Ringkasan:

Penjadwalan CPU bukan sekadar teori—ini adalah mesin di balik setiap sistem Linux yang responsif. Dengan menguasai algoritma seperti FCFS, SJF, RR, dan Prioritas, serta belajar menghitung metrik seperti Dengan mengurangi waktu tunggu dan waktu penyelesaian, Anda mendapatkan alat analisis untuk mengevaluasi kinerja sistem. dan mengambil keputusan berdasarkan informasi—persis seperti yang dipersyaratkan oleh RPS dan sistem dunia nyata. administrasi.



APLIKASI PRAKTIS

TUJUAN

PRAKTIK : Siswa secara langsung mencoba perintah dasar untuk mengamati, mengelola, dan men-debug proses di Rocky Linux.

(Latihan belum ditampilkan; siswa menunggu instruksi selama sesi latihan.)

DEMO - STUDI KASUS

Situasi

Anda adalah administrator sistem untuk laboratorium kampus. Seorang mahasiswa melaporkan dua masalah pada server Rocky Linux:

1. Proses latar belakang yang tidak diinginkan mengonsumsi CPU dalam jumlah tinggi.
2. Layanan **SSH** gagal untuk memulai ulang setelah perubahan konfigurasi—dan mereka tidak tahu penyebabnya.
untuk mendiagnosis kesalahan.

Tugas Anda: memulihkan sistem sepenuhnya hanya menggunakan perintah CLI—tanpa catatan modul, alat GUI, atau file yang dimuat sebelumnya.

TUGAS DEMO (Mahasiswa Harus Melakukannya Secara Langsung di Terminal)

1. Membuat Proses Latar Belakang dengan Penggunaan

CPU Tinggi • Simulasikan proses yang macet dengan menjalankan:

```
praktikumso@localhost:~$ yes > /dev/null &
```

- Perhatikan **nomor pekerjaan** dan **PID** yang ditampilkan (misalnya, `[1] 1234`).

2. Mengidentifikasi dan Mengamati Proses •

Gunakan `ps aux` atau `top` untuk menemukan proses `yes`. • Gunakan `pstree -p` untuk memastikan hubungan induk-anaknya. • Amati penggunaan CPU-nya di `top` atau `htop`.

3. Menghentikan Proses dengan Aman •

Kirim `SIGTERM` menggunakan `kill [PID]`. • Jika masih berjalan setelah 5 detik, kirim `SIGKILL` (`kill -9 [PID]`). • Verifikasi proses telah berhenti dengan:

```
praktikumso@localhost:~$ ps aux | grep yes
```

4. Simulasikan Kegagalan Layanan SSH

- Menambahkan baris yang tidak valid ke konfigurasi SSH:

```
praktikumso@localhost:~$ echo "port abc" | sudo tee -a /etc/ssh/sshd_config
```



- Mulai ulang SSHD
- Konfirmasikan bahwa proses gagal. Status sshd:

5. Mendiagnosis dan Memperbaiki

Masalah • Periksa log `sshd` (menggunakan `journalctl`) sejak 1 menit yang lalu • Identifikasi baris yang tidak valid (`Port abc`). • Hapus baris tersebut

`sudo sed -i '/Port abc/d' /etc/ssh/sshd_config` • Validasi sintaks: `sudo sshd -t`

- Mulai ulang SSH
- Verifikasi SSH aktif (petunjuk: `systemctl`)

6. Mendemonstrasikan Kontrol Pekerjaan (Opsional)

- Jalankan:

```
praktikumso@localhost:~$ sleep 60 &
```

- Daftar tugas: `jobs` •

Tunda dengan `Ctrl+Z`, lalu lanjutkan di latar belakang: `bg` • Hentikan dengan: `kill %1`

Aturan Demo

1. Tidak ada akses ke modul atau catatan.

Mahasiswa dilarang membuka PDF modul, catatan pribadi, atau dokumentasi online.

2. Semua Tugas Dilakukan Langsung di

Terminal Tidak ada file atau skrip yang dimuat sebelumnya—semuanya dibuat dan diperbaiki selama demo.

3. Observasi Langsung oleh Instruktur/Asisten Penilai akan mengevaluasi:

- Alur kerja logis (identifikasi → kontrol → verifikasi), • Penggunaan `SIGTERM` yang tepat sebelum `SIGKILL`, • Kemampuan untuk menginterpretasikan output `journalctl` dan memperbaiki konfigurasi.

4. Verifikasi Wajib Setiap langkah

harus dikonfirmasi menggunakan perintah seperti: • `ps`, `systemctl is-active`, `sshd -t`

5. Ikuti Instruksi Langsung Siswa

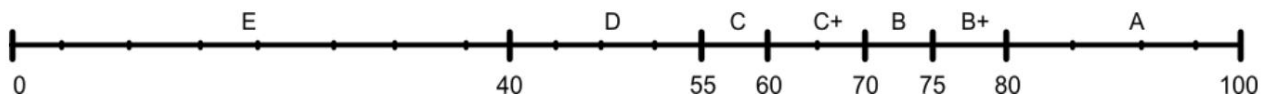
harus merespons secara interaktif arahan dari instruktur.



RUBRIK PENILAIAN

Aspek Penilaian	Poin (Total 100%)
LATIHAN	Total 5%
Latihan 1	1%
Latihan 2	1%
Latihan 3	1%
Latihan 4	1%
Latihan 5	1%
PRAKTIK	Total 20%
Latihan 1	10%
Latihan 2	10%
DEMO	Total 75%
Penyelesaian Studi Kasus	35%
Memahami Pemantauan & Analisis Proses	10%
Memahami Penguasaan Penanganan Sinyal	10%
Memahami Diagnostik Layanan & Log	10%
Memahami Kemampuan Pengendalian Pekerjaan	10%

SKALA PENILAIAN



A = (81 - 100) ÿ Sepuh

B+ = (75 - 80) ÿ Sangat Baik

B = (70 - 74) ÿ Baik

C+ = (60 - 69) ÿ Cukup Baik

C = (55 - 59) ÿ Adil

D = (41 - 54) ÿ Miskin

E = (0 - 40) ÿ Bro sungguh...

